

Exercice 1 :

```
#include <iostream>

#include <stdexcept> // pour std::runtime_error

using namespace std;

class Fraction {

    int num;

    int den;

    // Fonction pour calculer le PGCD (simplification des fractions)
    int pgcd(int a, int b) {
        return b == 0 ? a : pgcd(b, a % b);
    }

    // Normalisation de la fraction
    void normalise() {
        // Gestion du signe
        if (den < 0) {
            num = -num;
            den = -den;
        }

        // Simplification
        int div = pgcd(abs(num), den);
        if (div != 0) {
            num /= div;
            den /= div;
        }
    }

public:
    // Constructeurs
```

```
Fraction(int n = 0, int d = 1) {  
    num = n;  
    den = d;  
    if (d == 0) throw runtime_error("Dénominateur nul");  
    normalise();  
}
```

```
Fraction(const Fraction& f) {  
    num = f.num;  
    den = f.den;  
    normalise();  
}
```

// Opérateurs arithmétiques

```
Fraction operator+(const Fraction& f) {  
    return Fraction(num * f.den + f.num * den, den * f.den);  
}
```

```
Fraction operator-(const Fraction& f) {  
    return Fraction(num * f.den - f.num * den, den * f.den);  
}
```

```
Fraction operator*(const Fraction& f) {  
    return Fraction(num * f.num, den * f.den);  
}
```

```
Fraction operator/(const Fraction& f) {  
    return Fraction(num * f.den, den * f.num);  
}
```

// Affichage

```

void afficher(){
    cout<<num<<" / "<<den<<endl;
}

};

int main() {
    try {
        Fraction f1, f2;
        int num, den;
        // Saisie des fractions
        cout << "Entrez la premiere fraction (format a/b ou a): ";
        cout<<"Entrer num et den : ";
        cin>>num>>den;

        cout << "Entrez la seconde fraction (format a/b ou a): ";
        cout<<"Entrer num et den : ";
        cin>>num>>den;

        // Calcul de E = (f1 + 3/4 - f2)/(f1*f2 - 5/8) + 4
        Fraction trois_quarts(3, 4);
        Fraction cinq_huitiemes(5, 8);
        Fraction quatre(4);

        Fraction numerateur = f1 + trois_quarts - f2;
        Fraction denominateur = f1 * f2 - cinq_huitiemes;

        Fraction E = (f1 + trois_quarts - f2)/(f1 * f2 - cinq_huitiemes)+ quatre;
        //Fraction F = numerateur / denominateur + quatre;

        // Affichage du resultat

```

```

    cout<<"Fraction E = ";

    E.afficher();

    //cout<<"Fraction F = ";

    //F.afficher();

} catch (const exception& e) {

    cerr << "Erreur: " << e.what() << endl;

    return 1;

}

return 0;

}

```

Exercice 2 :

```

#include <iostream>

using namespace std;

#include <cstring>

```

```

class Ensemble {

private:

    int *tab;

    int taille; Ensemble e1, e2; => e1 + e2 ???

    int nbelts; // Nombre d'éléments dans l'ensemble

public:

    Ensemble(int a=10){

        taille = a;

        tab = new int[taille];

        nbelts = 0;

    }
}

```

```

Ensemble(const Ensemble &ens){
    taille = ens.taille;
    nbelts = ens.nbelts;
    tab = new int[taille];
    for(int i=0; i<nbelts; i++) {tab[i]=ens.tab[i];}
}

Ensemble operator=(const Ensemble&);
int operator>(int);    // Vérifie si un élément est présent
int operator>(const Ensemble&); // Vérifie si un ensemble est inclus
void operator+(int); // Retourne un nouvel ensemble avec l'élément ajouté
Ensemble operator+(const Ensemble&); // Retourne l'union de deux ensembles
int estvide();        // Vérifie si l'ensemble est vide
int cardinal();
void afficher();

friend ostream& operator<<(ostream& os, const Ensemble& ens);
};

Ensemble Ensemble::operator=(const Ensemble& ens) {
    if (this != &ens) {        // Éviter l'auto-affectation
        delete tab;           // Libérer l'ancien tableau

        taille = ens.taille;
        tab = new int[taille]; // Allouer un nouveau tableau
        nbelts = ens.nbelts;

        for (int i = 0; i < nbelts; i++) {
            tab[i] = ens.tab[i]; // Copier les éléments
        }
    }
    return (*this);
}

```

```

int Ensemble::operator>(int a){
    for (int i = 0; i < nbelts; ++i) {
        if (tab[i] == a)
            return 1;
    }
    return 0;
}

int Ensemble::operator>(const Ensemble &ens){
    for (int i = 0; i < ens.nbelts; ++i) {
        if (!(*this > ens.tab[i])) {
            return 0; // dès qu'un élément n'est pas trouvé
        }
    }
    return 1; // tous les éléments sont présents
}

void Ensemble::operator+(int a){
    int p = (*this) > a;
    if(p==1){
        cout<<"Est deja existant ! \n";
    }
    else{
        if (nbelts==taille){
            cout<<"Ensemble plein";
        }
        else{
            tab[nbelts]=a; // tab[nbelts++]=a;
            nbelts++;
        }
    }
}

```

```

Ensemble Ensemble::operator+(const Ensemble &ens){
    Ensemble res(taille + ens.taille);
    for(int i=0; i<nbelts; i++){
        res.tab[i] = tab[i];
    }
    res.nbelts = nbelts;
    for(int i=0; i<ens.nbelts; i++){
        res + ens.tab[i];
    }

    return res;
}

```

```

int Ensemble::estvide(){
    return(nbelts==0); // if(nbelts==0)return 1;
                        // else return 0;
}

```

```

int Ensemble::cardinal(){
    return nbelts;
}

```

```

void Ensemble::afficher(){
    cout << "{ ";
    for(int i=0; i<nbelts; i++){
        cout<<tab[i]<<" "<<"\n";
    }
    cout << "} ";
}

```

```

main() { // Ajouter programme principal ici }

```